

2) Algoritmi recursivi cu liste

(simplu soluție)

- Vom accesa un nr. de la începutul listei
- Nu avem nevoie de len. random
- Nu avem acces la lungimea listei dar putem verifica dacă e constantă ($n=0, n=1, \dots$)
- vom scrie funcții pure
- $\Rightarrow$ nu vom modifica datele de intrare
- $\Rightarrow$ vom crea liste noi
- vom obține la începutul listelor elem $\oplus l_1, l_2, \dots, l_n$
 $\cup$ $\underbrace{\hspace{1cm}}$
 în lista oșta obținu la început elem (notăție)

3) Îi se calculează produsul nr. impare dintr-o listă cu numere impare

. Dacă nu există nr. impare se va returna -1

$$\text{produs}(l_1, l_2, \dots, l_n, \text{cnt}) = \begin{cases} -1, & \text{dacă } \text{cnt} = 0 \text{ și } n = 0 \\ 1, & \text{dacă } \text{cnt} \neq 0 \text{ și } n = 0 \\ l_1 * \text{produs}(l_2, l_3, \dots, l_n, \text{cnt} + 1) & \text{dacă } l_1 \% 2 = 1 \\ \text{produs}(l_2, l_3, \dots, l_n), & \text{altfel} \end{cases}$$

$$\text{Produs_int}(l_1, l_2, \dots, l_n) = \text{produs}(l_1, l_n, 0)$$

În implementare presupunem că avem
 LISTAD cu urm. operații:

- $vide(lista): bool$
- $prim(lista): TElem$
- $sublista = lista - prim(lista): Listă$
 $\Leftrightarrow l_2, \dots, l_n$

- $adauga - inciput(lista, elem): Listă$

- $creste - vide(): Listă$

def $produs - impare(lista, cnt):$

if $cnt == 0$ and $vide(lista):$
 return -1

elif $cnt != 0$ and $vide(lista):$
 return 1

elif $prim(lista) \% 2 == 1:$

return $prim(lista) * produs - impare(sublista - fora - prim(lista), cnt)$

elif
 return $produs - impare(sublista - fora - prim(lista), cnt)$

4) Să se adauge o valoare elem pe
 poziția m ($m \geq 1$) într-o listă.

ex:

$l = [2, 3, 4]$, elem = 100, m = 3 ^{↓ poziție}

$[1, 2, 100, 3, 4]$

$m = 5 \Rightarrow [1, 2, 3, 4, 100]$

$m = 500 \Rightarrow [1, 2, 3, 4]$

$$(l_1, \dots, l_n, m, elem) = \begin{cases} \emptyset, & \text{dacă } m = 0, m > 1 \\ \text{dacă} \end{cases}$$

$$(l_1, \dots, l_n, m, \text{elem}) = \begin{cases} \emptyset, & n=0, m>1 \\ \text{elem} \oplus l_1, \dots, l_n, m-1 & \text{base} \\ l_1 \oplus \text{insert}(l_2, \dots, l_n, m, \text{elem}), \text{skip } l_1 \end{cases}$$